



МИНОБРНАУКИ РОССИИ

Федеральное государственное бюджетное образовательное учреждение  
высшего образования

«МИРЭА – Российский технологический университет»

**РТУ МИРЭА**

---

Институт информационных технологий

Кафедра математического обеспечения и стандартизации  
информационных технологий

## **ОТЧЁТ ПО ПРАКТИЧЕСКОЙ РАБОТЕ № 2**

по дисциплине «Тестирование и верификация  
программного обеспечения»

**Тема: «Модульное и мутационное тестирование  
программного продукта»**

Команда «Дерби» · Дата выполнения: 09.10.2026

Состав: Дубейко А. С., Домбровский Я. А., Егшатын Г. А.,

Яхьяев Ф. Ф. О., Лисенков А. М.

Выполнил:

Студент группы  
ИКБО-61-24

Лисенков Андрей Михайлович

Проверил:

Доцент кафедры МОСИТ,  
кандидат технических наук, доцент  
Чернов Е. А.

Москва 2026

## Содержание

|                                            |    |
|--------------------------------------------|----|
| 1 Цель и организация исследования.....     | 3  |
| 2 Приложение и контракты функций.....      | 4  |
| 3 Модульное тестирование.....              | 6  |
| 4 Учебный дефект и повторная проверка..... | 8  |
| 5 Покрытие строк и ветвей.....             | 9  |
| 6 Мутационное тестирование.....            | 10 |
| 7 Анализ результатов и выводы.....         | 12 |
| Список использованных источников.....      | 13 |
| Приложение А. Исследуемый код.....         | 14 |
| Приложение Б. Тесты и журналы.....         | 16 |
| Приложение В. Воспроизведение.....         | 19 |

Индивидуальное исследование звуковой подсистемы обновлённой игры. Полные материалы: experiments\_practice2\_lisenkov.zip.

## 1 Цель и организация исследования

Цель — проверить надёжность звуковой подсистемы игры «Дерби у стадиона», оценить качество утверждений модульных тестов и усилить их на основе новой выборки мутаций. Предмет исследования — доступность аудиоустройства, разрешение звука, синтез РСМ и сигналы игровых действий.

Задачи: описать шесть функций общего модуля game.py; подготовить независимый базовый набор тестов; внести учебный дефект; зарегистрировать его, исправить и повторить проверки; измерить покрытие; автоматически создать небольшие изменения кода, найти выжившие варианты и расширить тесты.

Таблица 1 — Организация работы

| Параметр               | Значение                                                                         |
|------------------------|----------------------------------------------------------------------------------|
| Автор отчёта           | Лисенков Андрей Михайлович                                                       |
| Группа / команда       | ИКБО-61-24 / «Дерби»                                                             |
| Состав команды         | Дубейко А. С., Домбровский Я. А., Егшатын Г. А., Яхьяев Ф. Ф. О., Лисенков А. М. |
| Индивидуальная область | Звук и связанное управление в game.py, 6 методов                                 |
| Язык и инструменты     | Python; pytest, Mock, coverage.py; автоматический генератор мутаций              |
| Дата / преподаватель   | 09.10.2026 / Чернов Е. А.                                                        |

Выполнено индивидуальное исследование общего модуля игры с выбором клубов. Для звуковой подсистемы отдельно подготовлены тесты, учебная дефектная копия, новая выборка мутаций и журналы автоматических запусков.

В изолированных копиях воспроизведён цикл «внесение ошибки → проверка → карточка дефекта → исправление → повторная проверка». Карточка описывает результат и исправление; журналы сохраняют доказательства автоматических проверок.

## 2 Приложение и контракты функций

Игра позволяет выбрать обе стороны из десяти испанских клубов; по умолчанию болельщики «Барселоны» бросают предметы в персонажа «Реал Мадрид». Space запускает бросок, 1–3 выбирают предмет, T открывает меню, M переключает звук, R сбрасывает счёт.

Таблица 2 — Контракты шести методов

| Метод             | Ожидаемое поведение                                                                                  |
|-------------------|------------------------------------------------------------------------------------------------------|
| Sound.__init__    | Успех: available=True, enabled=True, эффекты throw и hit. Ошибка mixer/Sound: оба флага False.       |
| Sound.play(name)  | Воспроизводит только указанный эффект и только когда enabled и available истинны.                    |
| Sound.toggle      | При available=True инвертирует enabled. При available=False сохраняет значение enabled.              |
| Game.throw        | Вызывает state.throw(); сигнал throw только при успешном броске.                                     |
| Game.handle_event | M переключает звук в игре, блокируется в меню. R не меняет предпочтение звука.                       |
| Game.step(dt)     | Сначала события, затем update вне меню; один сигнал hit при ненулевом результате update; затем draw. |

enabled означает пользовательское разрешение, available — успешную инициализацию устройства. Это разные признаки. Корректность требует проверки всех четырёх сочетаний: воспроизведение допускается только при (True; True).

Параметры синтеза зафиксированы как регрессионный контракт текущей версии: 22050 Гц, знаковый PCM 16 бит, один канал; throw — 0,12 с, hit — 0,18 с. Они установлены по текущему коду и не выдаются за отдельно заданные пользователем требования.

## 2.1 Контрольный запуск приложения

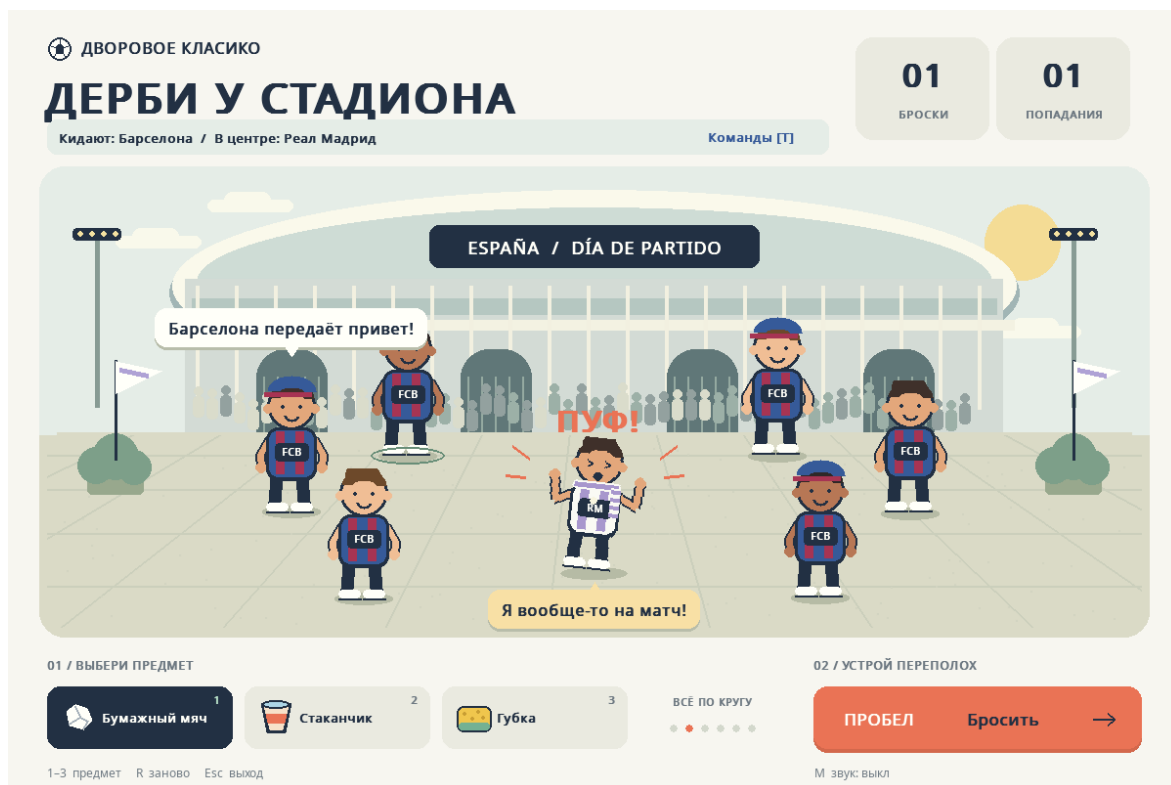


Рисунок 1 — Реальный кадр после броска при выключенном звуке

Изображение получено при отдельном запуске интеграционного сценария в режиме SDL dummy. В нём М выключает звук; бросок и попадание сохраняют игровую логику; М в меню игнорируется; после закрытия меню повторное М включает звук.

Интеграционный сценарий создаёт настоящий объект Game и использует настоящий dummy mixer. Это дополнение к модульным проверкам с Mosk. Снимок доказывает выполнение отрисовки; наличие или качество слышимого звука по изображению не оценивается.

### 3 Модульное тестирование

Базовый набор содержит десять случаев: успешная инициализация, отказ mixer, доступный и недоступный переключатель, разрешённое воспроизведение, полное отключение, клавиша M, успешный бросок, попадание и отсутствие попадания. Каждая проверка получает новое состояние.

Модульные тесты подменяют pygame.mixer.init и pygame.mixer.Sound. Конструктор Sound при этом действительно генерирует PCM; mock сохраняет переданный буфер. Для управления создаётся Game.\_\_new\_\_, состояние и отрисовка заменяются Mock. В отдельной проверке шага используется настоящий Sound с подменённым mixer.

Таблица 3 — Результаты исправного кода

| Набор      | Случаев | Провалов | Ошибок | Пропусков |
|------------|---------|----------|--------|-----------|
| Базовый    | 10      | 0        | 0      | 0         |
| Улучшенный | 39      | 0        | 0      | 0         |

Число случаев включает параметры pytest; здесь нет дополнительных unittest subtests. Источники — summary.json и JUnit. Числа из прежних работ не добавляются к этим результатам.

Таблица 4 — Среда измерения

| Компонент         | Значение                                                               |
|-------------------|------------------------------------------------------------------------|
| Python            | 3.12.14                                                                |
| Зависимости       | pytest 9.1.1; coverage 7.16.2; pygame-ce 2.5.8                         |
| Драйверы          | SDL_VIDEODRIVER=dummy; SDL_AUDIODRIVER=dummy                           |
| Изоляция          | Отдельный временный каталог и процесс для каждого варианта             |
| Версия исходников | Исправные src/game.py и src/model.py; SHA-256 сохранены в summary.json |

### 3.1 Усиление модульных проверок

Таблица 5 — Дополнительные группы проверок

| Область                    | Проверка результата                                                                                         |
|----------------------------|-------------------------------------------------------------------------------------------------------------|
| Формат и длительность      | Аргументы mixer.init; тип массива h; 2646 отсчётов throw и 3969 отсчётов hit.                               |
| Форма РСМ                  | Ненулевой сигнал обеих полярностей; пик от 4000 до 6500; первый отсчёт 0; ранний пик <2000; финальный <100. |
| Различимость эффектов      | Первые 200 отсчётов throw и hit различаются.                                                                |
| Разрешение воспроизведения | Два эффекта × четыре сочетания enabled/available; правильный эффект вызван ровно один раз либо ни разу.     |
| Переключение               | Двойное toggle возвращает исходное состояние; без устройства состояние сохраняется.                         |
| Отказ аудио                | Ошибка при создании самого Sound безопасно отключает подсистему.                                            |
| Связь с игрой              | Отказ броска не звучит; 0/1/3 попадания дают 0/1/1 вызов hit; меню блокирует M, Space, R и update.          |
| Порядок событий            | M и попадание в одном кадре: сначала отключение, затем отсутствие звучания hit; R сохраняет отключение.     |

Характеристики РСМ описывают регрессию текущей версии. Тесты не пересчитывают её синусоидальную формулу: проверяются наблюдаемые свойства массива и отсутствие тишины, резкого начала или незатухающего хвоста. Пороги оставляют диапазон допустимых отсчётов.

Все десять базовых случаев сохранены. Ещё 29 параметризованных случаев уточняют результат и отсутствие лишних действий. Дополнения связаны с причинами выживания мутаций, перечисленными в разделе 6.

## 4 Учебный дефект и повторная проверка

Таблица 6 — Карточка BUG-SOUND-01

| Поле               | Содержание                                                                                                                             |
|--------------------|----------------------------------------------------------------------------------------------------------------------------------------|
| Краткое описание   | Переключатель устанавливает <code>enabled=True</code> при недоступном аудиоустройстве.                                                 |
| Расположение       | <code>game.py</code> , <code>Sound.toggle</code> ; условие <code>self.available</code> заменено на <code>True</code> .                 |
| Категория / статус | Minor: неверное состояние настройки без остановки игры. Open → Fixed → Closed.                                                         |
| Предусловия        | <code>available=False</code> , <code>enabled=False</code> после отказа аудиоустройства.                                                |
| Шаги               | 1. Подготовить недоступный <code>Sound</code> .<br>2. Вызвать <code>toggle</code> (действие M).<br>3. Проверить <code>enabled</code> . |
| Ожидание / факт    | Ожидается <code>enabled=False</code> ; в дефектной копии получено <code>True</code> .                                                  |
| Причина            | Удалена проверка доступности; флаг инвертируется безусловно.                                                                           |
| Исправление        | Восстановить <code>if self.available</code> перед инверсией <code>enabled</code> .                                                     |

До исправления: 1 провал из 10 случаев; после исправления — 10 успешно, без ошибок и пропусков. Сработал `test_toggle_unavailable_does_not_enable_audio`.

Учебная ошибка:  

```
if True:
    self.enabled = not self.enabled
```

Исправленный фрагмент:  

```
if self.available:
    self.enabled = not self.enabled
```

Дефект внесён в отдельную учебную копию. При исправлении восстановлен исходный `guard`, затем повторён тот же базовый набор. Опубликованная игра этой ошибкой не изменялась.

## 5 Покрывание строк и ветвей

Покрывание строк и покрывание ветвей приводятся отдельно. coverage.py запускается с --branch и объединяет выполненные участки, а не средние проценты. Срез Sound включает три метода; срез шести методов дополнительно включает управление из таблицы 2.

Таблица 7 — Фактическое покрывание

| Область / набор           | Строки           | Ветви            |
|---------------------------|------------------|------------------|
| Sound, 3 метода; База     | 20/20 (100,00%)  | 8/8 (100,00%)    |
| Sound, 3 метода; Усиление | 20/20 (100,00%)  | 8/8 (100,00%)    |
| Sound, 3 метода; + SDL    | 20/20 (100,00%)  | 8/8 (100,00%)    |
| 6 методов; База           | 36/66 (54,55%)   | 21/54 (38,89%)   |
| 6 методов; Усиление       | 40/66 (60,61%)   | 26/54 (48,15%)   |
| 6 методов; + SDL          | 55/66 (83,33%)   | 41/54 (75,93%)   |
| весь game.py; База        | 89/462 (19,26%)  | 22/180 (12,22%)  |
| весь game.py; Усиление    | 93/462 (20,13%)  | 27/180 (15,00%)  |
| весь game.py; + SDL       | 414/462 (89,61%) | 148/180 (82,22%) |

Главное наблюдение: Sound уже в базовом наборе имеет 100% строк и ветвей, однако шесть из двенадцати мутаций выживают. Выполнение строки генерации РСМ ничего не говорит о том, проверен ли переданный буфер. Прохождение обоих исходов and не заменяет полную таблицу комбинаций.

Неполное покрывание общего game.py объясняется областью эксперимента: в него входят сцена, выбор клубов, координаты мыши, цикл запуска и обработка исключений. 100% всего файла не заявляется. Полные пропущенные строки и ветви доступны в coverage.json и htmlcov/.

## 6 Мутационное тестирование

Для новой серии задано 12 одношаговых замен L01–L12 в game.py. Собственный ограниченный генератор проверяет единственность исходного фрагмента, заменяет его, разбирает AST и запускает тесты в свежем процессе. Изменённые файлы и описания замен сохраняются рядом с журналами.

Сначала проверяется исправный код. Мутант считается обнаруженным только при реальном провале утверждений. Ошибки сбора тестов, исключения запуска и тайм-ауты учитываются отдельно и не повышают результат. Лимит каждого процесса — 40 с; таких ошибок в этой серии не было.

Таблица 8 — Новая выборка мутаций

| ID  | Изменение                                                | База      | Усиление  |
|-----|----------------------------------------------------------|-----------|-----------|
| L01 | Частота mixer удвоена без изменения PCM                  | Выжил     | Обнаружен |
| L02 | Длительность броска сокращена вдвое                      | Выжил     | Обнаружен |
| L03 | Генератор формирует только нулевые отсчёты               | Выжил     | Обнаружен |
| L04 | Успешная инициализация оставляет звук недоступным        | Обнаружен | Обнаружен |
| L05 | После сбоя mixer звук помечается включённым              | Обнаружен | Обнаружен |
| L06 | Одного разрешающего флага достаточно для воспроизведения | Выжил     | Обнаружен |
| L07 | Вместо запрошенного эффекта всегда звучит бросок         | Выжил     | Обнаружен |
| L08 | Переключатель всегда включает звук                       | Обнаружен | Обнаружен |
| L09 | М воспроизводит сигнал вместо переключения               | Обнаружен | Обнаружен |
| L10 | Бросок воспроизводит эффект попадания                    | Обнаружен | Обнаружен |
| L11 | Попадание воспроизводит эффект броска                    | Обнаружен | Обнаружен |
| L12 | Удалено плавное нарастание сигнала                       | Выжил     | Обнаружен |

В выборке используются изменения констант, замена логической операции и удаление множителя. Учебный BUG-SOUND-01 учитывается отдельно и не включён в 12 вариантов.

## 6.1 Анализ выживших вариантов

Таблица 9 — Причина выживания и устранение пробела

| ID  | Причина                                               | Новая проверка                                                             |
|-----|-------------------------------------------------------|----------------------------------------------------------------------------|
| L01 | Не проверялись аргументы инициализации mixer.         | Проверить частоту 22050, формат -16, один канал и buffer=512.              |
| L02 | Достаточно было наличия ключа throw в словаре.        | Проверить длину РСМ для обоих эффектов.                                    |
| L03 | Содержимое массива могло полностью состоять из нулей. | Проверить ненулевую амплитуду и обе полярности.                            |
| L06 | Проверялись только True/True и False/False.           | Проверить все четыре комбинации и отсутствие вызовов при смешанных флагах. |
| L07 | Вызов play был проверен только для имени throw.       | Запросить hit и убедиться, что вызван именно hit, а throw молчит.          |
| L12 | Не проверялась мягкая атака звука.                    | Проверить предел амплитуды в первом проценте буфера.                       |

Мутационная оценка = число обнаруженных / число валидных мутантов  $\times 100\%$ . До усиления:  $6/12 = 50,00\%$ . После:  $12/12 = 100,00\%$ . Повторно используется та же выборка, поэтому результаты сопоставимы.

Все шесть выживших вариантов имеют наблюдаемое отличие и после добавления соответствующего утверждения обнаруживаются. Эквивалентные варианты в этой выборке не заявлены. Достижение 100% относится только к выбранным 12 заменам, а не ко всем возможным ошибкам программы.

## 7 Анализ результатов и выводы

Новая серия подтвердила исправность проверенных сценариев работы звука на исходной версии: базовый набор 10/10, усиленный 39/39. Учебный дефект доступности переключателя воспроизводимо обнаружен; после восстановления условия тот же набор проходит полностью.

Мутационный анализ выявил два класса недостатков базовых тестов: не проверялся результат генерации РСМ и были пропущены отдельные сочетания условий и имён эффектов. Добавленные проверки устранили оба пробела: оценка выбранной серии выросла с 50% до 100%.

Высокое покрытие Sound само по себе оказалось недостаточным: при 20/20 строк и 8/8 ветвей часть неправильных вариантов оставалась незамеченной. Поэтому оценка качества включает не только выполнение кода, но и проверку результата, параметров вызова и отсутствия лишних действий.

Взаимодействие с игрой проверено отдельно: отклонённый бросок молчит; при нуле попаданий сигнал не вызывается; несколько попаданий за один update дают один сигнал; открытое меню приостанавливает обновление и блокирует M; отключение в начале кадра действует на последующее попадание.

Ограничения: проверяется выбранная область общего модуля, а не все функции игры. Mock и dummy mixer дают воспроизводимость, но не оценивают реальное устройство macOS, громкость, задержку, искажения и субъективное качество. Для этих свойств необходим дополнительный ручной запуск на оборудовании.

Комплект содержит исходники, все тесты, преднамеренную ошибку, 12 правил мутаций, журналы, JUnit и HTML-покрытие. Числа в отчёте получены из новых запусков; предыдущие результаты не использованы как измерения этой работы.

## **Список использованных источников**

1. Практическая работа № 2. Модульное и мутационное тестирование программного продукта. Предоставленные методические указания, разделы 2.1–2.4.
2. Предоставленный шаблон «Титульный лист\_практик (1).docx». Институт информационных технологий РТУ МИРЭА.
3. ГОСТ 19.401–78. ЕСПД. Текст программы. Требования к содержанию и оформлению.
4. ГОСТ 34.602–2020. Информационная технология. Комплекс стандартов на автоматизированные системы. Техническое задание на создание автоматизированной системы.
5. pytest. Documentation. <https://docs.pytest.org/> (дата обращения: 09.10.2026).
6. coverage.py. Branch coverage measurement. <https://coverage.readthedocs.io/> (дата обращения: 09.10.2026).
7. Python. unittest.mock. <https://docs.python.org/3/library/unittest.mock.html> (дата обращения: 09.10.2026).
8. pygame-ce. Mixer documentation. <https://pyga.me/docs/ref/mixer.html> (дата обращения: 09.10.2026).
9. Исходный проект «Дерби у стадиона». <https://github.com/akimdubeyko-dotcom/calculator>. Зафиксированные файлы и SHA-256 входят в материалы эксперимента.

## Приложение А. Исследуемый код

Листинг А.1 — звуковая подсистема и сигналы игровых действий. Фрагменты из исправного src/game.py; полные исходники обоих модулей включены в архив.

```
class Sound:
    def __init__(self):
        self.enabled = True
        self.available = False
        self.sounds = {}
        try:
            pygame.mixer.init(frequency=22050, size=-16, channels=1, buffer=512)
            for name, frequency, duration in [("throw", 650, 0.12), ("hit", 170, 0.18)]:
                samples = array("h")
                count = int(22050 * duration)
                for i in range(count):
                    t = i / count
                    phase = math.tau * frequency * duration * (t - 0.35*t*t)
                    samples.append(int(6500 * math.sin(phase) * (1-t) ** 2 * min(t*30, 1)))
                self.sounds[name] = pygame.mixer.Sound(buffer=samples)
            self.available = True
        except pygame.error:
            self.enabled = False

    def play(self, name):
        if self.enabled and self.available:
            self.sounds[name].play()

    def toggle(self):
        if self.available:
            self.enabled = not self.enabled

    def throw(self):
        if self.state.throw():
            self.sound.play("throw")

    def step(self, dt):
        for event in pygame.event.get():
            self.handle_event(event)
        if not self.team_menu and self.state.update(dt):
            self.sound.play("hit")
        self.draw()
```

В контрольном BUG-SOUND-01 изменяется только условие `Sound.toggle`. Правила мутаций описаны в `run_experiments.py` и сохранены для каждого варианта в `mutation.json`.

## Приложение А — обработка пользовательских событий

```
def handle_event(self, event):
    if event.type == pygame.QUIT:
        self.running = False
    elif event.type == pygame.KEYDOWN:
        if event.key == pygame.K_t:
            self.team_menu = not self.team_menu
            return
        if self.team_menu:
            if event.key in (pygame.K_ESCAPE, pygame.K_RETURN):
                self.team_menu = False
            return
        if event.key == pygame.K_ESCAPE:
            self.running = False
        elif event.key == pygame.K_SPACE:
            self.throw()
        elif event.key in (pygame.K_1, pygame.K_2, pygame.K_3):
            self.state.select(event.key-pygame.K_1)
        elif event.key == pygame.K_r:
            self.state.reset()
        elif event.key == pygame.K_m:
            self.sound.toggle()
    elif event.type == pygame.MOUSEBUTTONDOWN and event.button == 1:
        pos = self.logical_mouse(event.pos)
        if self.team_menu:
            if TEAM_DONE.collidepoint(pos):
                self.team_menu = False
            else:
                for role, index, rect in TEAM_CHOICES:
                    if rect.collidepoint(pos):
                        self.state.choose_club(role, index)
                return
        if TEAM_BUTTON.collidepoint(pos):
            self.team_menu = True
            return
        if BUTTON.collidepoint(pos):
            self.throw()
        for i, rect in enumerate(ITEM_RECTS):
            if rect.collidepoint(pos):
                self.state.select(i)
    elif event.type == pygame.VIDEORESIZE:
        self.window = pygame.display.set_mode((max(480, event.w), max(320, event.h)),
pygame.RESIZABLE)
```

## Приложение Б. Тесты и журналы

Листинг Б.1 — проверки регрессионных свойств РСМ и таблицы разрешения.

Используются фикстуры `sound` и `mixer` из `conftest.py`; `mixer` сохраняет переданные буферы.

```
def test_mixer_format_matches_pcm(sound, mixer):
    mixer[0].assert_called_once_with(frequency=22050, size=-16, channels=1, buffer=512)

@pytest.mark.parametrize('index,duration', [(0, 0.12), (1, 0.18)])
def test_pcm_count(sound, mixer, index, duration):
    samples = mixer[1].call_args_list[index].kwargs['buffer']
    assert samples.typecode == 'h'
    assert len(samples) == int(22050*duration)

@pytest.mark.parametrize('index', [0,1])
def test_pcm_amplitude_and_envelope(sound, mixer, index):
    samples = mixer[1].call_args_list[index].kwargs['buffer']
    # Regression contract: audible signed PCM, soft attack, fading final segment.
    peak = max(map(abs, samples))
    assert 4000 < peak < 6500
    assert max(samples) > 0 and min(samples) < 0
    assert samples[0] == 0
    assert max(map(abs, samples[:len(samples)//100])) < 2000
    assert max(map(abs, samples[-len(samples)//10:])) < 100

def test_effects_have_distinct_pcm(sound, mixer):
    a, b = [call.kwargs['buffer'] for call in mixer[1].call_args_list]
    assert list(a[:200]) != list(b[:200])

@pytest.mark.parametrize('name', ['throw','hit'])
@pytest.mark.parametrize('enabled,available', [(True,True),(True,False),(False,True),
(False,False)])
def test_play_truth_table(sound, name, enabled, available):
    sound.enabled, sound.available = enabled, available
    sound.play(name)
    for key, effect in sound.sounds.items():
        assert effect.play.call_count == int(enabled and available and key == name)
```

## Приложение Б — отрицательные сценарии и порядок событий

```
@pytest.mark.parametrize('initial', [True, False])
def test_unavailable_toggle_preserves_state(sound, initial):
    sound.enabled, sound.available = initial, False
    sound.toggle()
    assert sound.enabled is initial

def test_rejected_throw_has_no_sound(controller):
    controller.state.throw.return_value = False
    controller.throw()
    controller.sound.play.assert_not_called()

def test_menu_step_does_not_update_or_play(controller, monkeypatch):
    monkeypatch.setattr(pygame.event, 'get', lambda: [])
    controller.team_menu = True
    controller.step(.5)
    controller.state.update.assert_not_called()
    controller.sound.play.assert_not_called()
    controller.draw.assert_called_once_with()

def test_m_toggle_applies_before_hit_in_same_frame(controller, sound, monkeypatch):
    controller.sound = sound
    event = pygame.event.Event(pygame.KEYDOWN, key=pygame.K_m)
    monkeypatch.setattr(pygame.event, 'get', lambda: [event])
    controller.state.update.return_value = 1
    controller.step(.1)
    assert sound.enabled is False
    sound.sounds['hit'].play.assert_not_called()
```

Фикстуры и остальные проверки доступны в tests/improved/. Исходный базовый набор находится отдельно в tests/baseline/. Каждый вариант запускается с одной из этих папок без импорта результатов другой серии.

## Приложение Б — фактические журналы запусков

### Корректный базовый набор

```
..... [100%]  
10 passed in 0.06s
```

### Дефектная учебная копия

```
...F..... [100%]  
===== FAILURES =====  
_____ test_toggle_unavailable_does_not_enable_audio _____  
tests/test_sound.py:34: in test_toggle_unavailable_does_not_enable_audio  
    assert sound.enabled is False  
E   assert True is False  
E   + where True = <game.Sound object at 0x7efe48db7e00>.enabled  
===== short test summary info =====  
FAILED tests/test_sound.py::test_toggle_unavailable_does_not_enable_audio - a...  
1 failed, 9 passed in 0.04s
```

### После исправления

```
..... [100%]  
10 passed in 0.03s
```

### Улучшенный корректный набор

```
..... [100%]  
39 passed in 0.21s
```

### Мутант L06: базовый набор

```
..... [100%]  
10 passed in 0.03s
```

### Мутант L06: улучшенный набор

```
===== short test summary info =====  
FAILED tests/test_extra.py::test_play_truth_table[True-False-throw] - Asserti...  
FAILED tests/test_extra.py::test_play_truth_table[True-False-hit] - Assertion...  
FAILED tests/test_extra.py::test_play_truth_table[False-True-throw] - Asserti...  
FAILED tests/test_extra.py::test_play_truth_table[False-True-hit] - Assertion...  
FAILED tests/test_extra.py::test_m_toggle_applies_before_hit_in_same_frame - ...  
5 failed, 34 passed in 0.17s
```

Показаны заключительные строки реальных журналов. Полные трассировки, JUnit и определения изменённых файлов находятся в соответствующих каталогах results/.

## Приложение В. Воспроизведение

Распаковать experiments\_practice2\_lisenkov.zip. Открыть Терминал в каталоге с run\_experiments.py. Для macOS с установленным Python 3.15 выполнить последовательно:

```
python3.15 -m venv .venv  
.venv/bin/python -m pip install -r requirements.txt  
.venv/bin/python run_experiments.py
```

Скрипт автоматически выполняет базовый набор, дефект до/после, 12 базовых мутантов, усиленный набор, те же 12 мутантов и интеграционные проверки. Рабочие файлы игры не перезаписываются; результаты создаются в каталоге этой лаборатории.

Таблица В.1 — Проверяемые материалы

| Путь                                | Назначение                             |
|-------------------------------------|----------------------------------------|
| src/                                | Исправные game.py и model.py           |
| tests/                              | Два набора и их фикстуры               |
| run_experiments.py                  | Автоматизация, BUG-SOUND-01, L01–L12   |
| integration_probe.py                | М, бросок, попадание, пауза, включение |
| results/summary.json                | Числа, версии, SHA-256                 |
| results/BUG-SOUND-01-*/             | Журналы дефекта и исправления          |
| results/baseline-L*/ и improved-L*/ | Мутанты, mutation.json, stdout, JUnit  |
| results/*/coverage.json и htmlcov/  | Строки и ветви                         |
| results/integration/                | Контрольные журналы и изображения      |

Результаты должны совпасть по числу случаев и статусам; время, временные пути и шрифты могут различаться. Реальные динамики не нужны. Подробные ограничения и карта материалов приведены в README лаборатории.